Task oversight: one list I can trust

A local app that pulls tasks from two AI assistants, a task manager and notes on the local PC into one prioritised view, built in phases from job stories and hardened before it was trusted.

Rory McClelland · mcclelland.info

Version 0.6 · October 2026 · code at github.com/rorymcclelland/task-manager

5
sources in one list: Claude, Gemini, Lunatask, notes and typed tasks; nothing is written back

18
structural decisions, each written down with its reason

13 of 13
QA findings fixed and retested, including one high-severity issue

154
automated tests, on Node's built-in runner

My tasks had spread across Claude, Gemini, Lunatask and notes on the local PC. Each list was fine on its own, but none was complete, so there was no single place that could answer "what should I be doing now?" with confidence. This app is that place. It pulls every source into one list, ranks it on the Eisenhower matrix I already think in, sets it against the calendar, and tells me plainly when the list has stopped being trustworthy.

It is a small product, and that is the point of this case study. The same disciplines I would expect of a team (job stories, a phased backlog, written decisions, a full QA pass and a hardening phase) were applied to a tool for one user. I owned the product and made every decision; Claude, as an AI coding agent, wrote the code, and I read and questioned every change. The app itself uses no AI. Its job is to keep track of the work that AI assistants create.

01

The problem, and the frame

Tasks were landing in four places, each for a different reason:

Source	What lands there
Claude	Work started or promised in conversations, and follow-ups
Gemini	The same, in a second assistant
Lunatask	The deliberate, curated task list and the calendar
The local PC	Notes and half-finished work that imply tasks

The scope was settled before any code was written: **one user**, with no sharing or collaboration; **Eisenhower prioritisation** built into the data model rather than added as a tag; and only the parts of a task manager that are actually used day to day. Notes, habits, mood tracking and contacts were ruled out by name. The data is personal, and some of it comes from other systems' accounts, so a never list was agreed too.

Never
write back to Lunatask, an assistant or a notes file

Never
delete a task; done and dropped stay on file

Never
merge two tasks without being asked

Never
read conversations; capture is deliberate

02

How it works

Everything runs on the local PC: a small Node.js server and a page in the browser at localhost. Tasks are kept in one file. Dashed steps are ordinary code; the dark step is the user.

Where tasks start
each one stays the master of its own tasks

Lunatask
Over its API, every 15 minutes

Claude and Gemini
A pasted block, or a file in an inbox folder

Notes on the PC
Checkboxes and TODO lines

The calendar
Its private .ics address; events, not tasks

The app, on the local PC
reads its sources, never writes back to them

1 Capture and sync
One list from five sources
Each task keeps a link back to where it came from. Likely duplicates are suggested, never merged on their own.

2 Worked out, not stored
Priority and honesty
Only the user's two answers (important? urgent?) and the dates are saved, in one JSON file. Each time the page asks, code works out the quadrant, raises urgency as a due date nears, flags an overfull "Do now", marks idle tasks and checks every source's health.

The user

3 Every day
The user decides
Matrix, Day, Ahead and List views. Drag or press 1–4 to re-prioritise; drag onto the calendar to give a task a time. A one-card-at-a-time review clears what needs deciding: unplaced, overdue, missed and idle tasks.

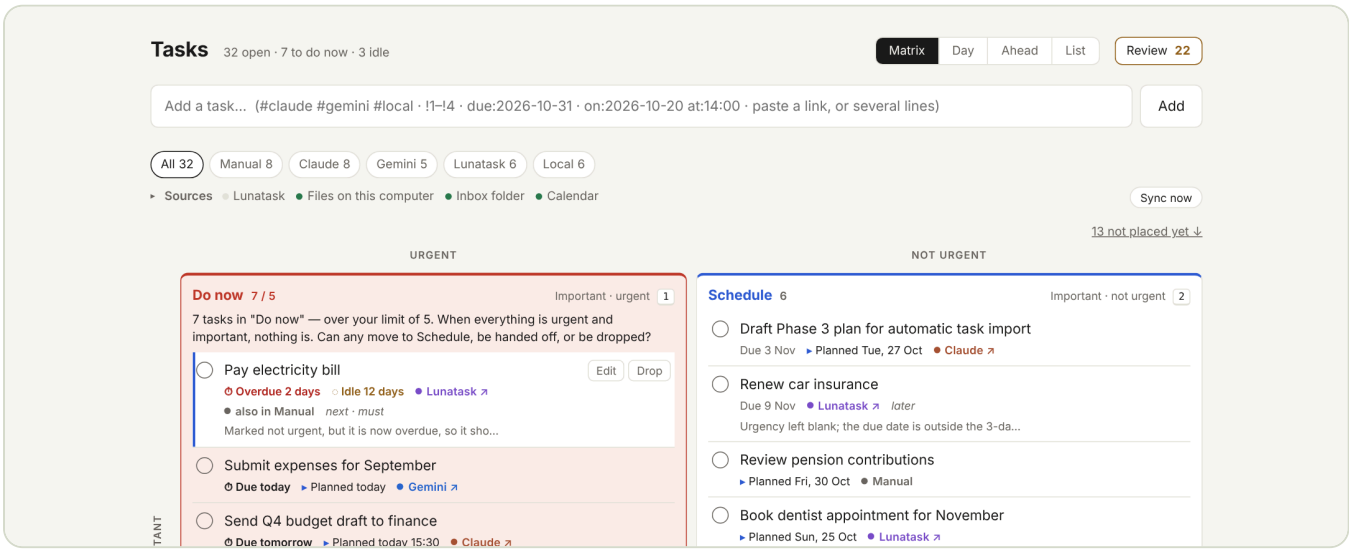
Changes save at once; complete and drop can be undone.

Stays on the local PC
- the task file
- the Lunatask token and calendar address
- calendar events, held in memory only

What flows in
- tasks, read-only, from each source
- status and dates from Lunatask
- whatever the user deliberately pastes or drops

Never happens
- writing back to any source
- deleting a task, or overwriting an unreadable file
- any other website changing a task

How the app fits together. Sources on the left are read, never written to; the app works out priority each time rather than storing it; the user works in four views and a review. The panels at the foot are the data boundaries and the never list.



The matrix, the default view, shown with made-up sample data. Each task shows where it came from and why it sits where it does; "Do now" is over its limit, so it says so.

03

Six decisions that shaped it

Each decision is numbered and kept. A reversed decision gets a new entry, not an edit.

Every structural choice was written down with its reason before or as it was built, so it can be questioned on its merits later. Eighteen were recorded in all; these six say most about the approach.

D8 · Sync direction

Every source is read-only

Nothing this app does is written back. Completing a Lunatask task here does not complete it in Lunatask, a known gap accepted on purpose: the first version earns trust by being unable to damage the systems it reads.

D9 · Capture

Deliberate capture, not scraping

Tasks from Claude and Gemini arrive only when someone sends them: a pasted block, or a file dropped in an inbox folder. The app never reads conversations, so it never holds more than it was given.

D5 · Priority

Worked out, not stored

The file holds only the user's answers and dates; the quadrant is calculated from them and today's date. Time passing changes priorities without anything rewriting the file in the background.

D4 · Duplicates

Suggest, never auto-merge

Two tasks are suggested as duplicates when they share at least 60% of their meaningful words. The rule fits in one sentence, so an odd suggestion explains itself, and merging waits for the user.

D7 · Lunatask

Respect the source's encryption

Lunatask encrypts task names end to end, so its API never returns them. Rather than work around that, each Lunatask task is named here once, by the user, and its status and dates stay in sync.

D2 · Storage

One file, safe in a sync folder

Tasks live in one readable JSON file, saved by writing a copy and swapping it in, so a sync client never sees half a save. Storage sits behind a small interface, so a database can replace it later without rewriting the app.

04

Testing: a QA pass before trusting it

After Phase 5, with every feature working, the app had a full QA pass of the kind a product would get before launch: the automated tests in four time zones (UTC, Dublin, Taipei and Los Angeles), a type check, a 40-step scripted browser walkthrough, an accessibility scan, and deliberate attempts to break it with bad dates, damaged files, odd encodings and requests from other websites.

Severity	Found	Examples
High	1	Any website open in the browser could add, complete or drop tasks through the running app.
Medium	5	Impossible dates such as 30 February accepted; an unreadable task file reported as a server fault; files saved from older PowerShell turned into junk tasks; light-mode labels below the 4.5:1 contrast standard; the duplicate check slowing to 3.8 seconds at 1,666 open tasks.
Low	6	One malformed task stopping the page loading, stray list lines setting a quadrant, a view that didn't move on at midnight, and smaller issues.

The serious one

The high-severity finding is the kind that matters most in a tool that runs quietly in the background. A test page served from another address planted a task in "Do now", because browsers let any page send certain simple requests without asking, and the server accepted them. A related trick, DNS rebinding, could have let a website read the list. The fix is three checks on every request: changes must arrive as JSON, which other sites cannot send without the browser asking first; a request naming another site as its origin is refused; and anything not addressed to the local PC itself is refused, reads included. Scripts on the PC, including a Claude session dropping in tasks, still work.

Hardening, then a retest

Every finding became a backlog item with a "done when" test, and Phase 6 fixed all twelve, each with an automated test that failed before the fix. The retest ran the same probes again. It confirmed every fix and found one more issue: after visiting one view, its content stayed on screen under the others. That was fixed and checked the same day. Thirteen found, thirteen fixed, and the retest written up alongside the original report.

05

How it was delivered

The backlog is written as job stories ("When tasks are scattered across systems, I want to see them all in a single list, so I can stop holding the gaps in my head"), grouped into phases that are each usable on their own, so the app earned its place from Phase 1 rather than at the end. Phases 0 to 6 were built, tested and hardened in a single day of focused work, with at least one reviewed commit per phase.

Phase	What it delivered
0 · Foundations	The stack decision, a data model able to hold a task from any source, and storage behind an interface.
1 · One list I can trust	One list, quick capture with shortcuts, complete and drop with Undo, and duplicate suggestions.
2 · Prioritisation	The Eisenhower matrix as the default view, due dates raising urgency automatically, and the "Do now" limit.
3 · Pulling in the sources	Lunatask over its API, Claude and Gemini by paste or inbox file, and notes on the PC.
4 · Time	The calendar beside the list, time slots, and the Day and Ahead views.
5 · Keeping it honest	Idle flags, the one-card review, and banners when a sync fails or stalls.
6 · Hardening	Every QA finding fixed and retested.

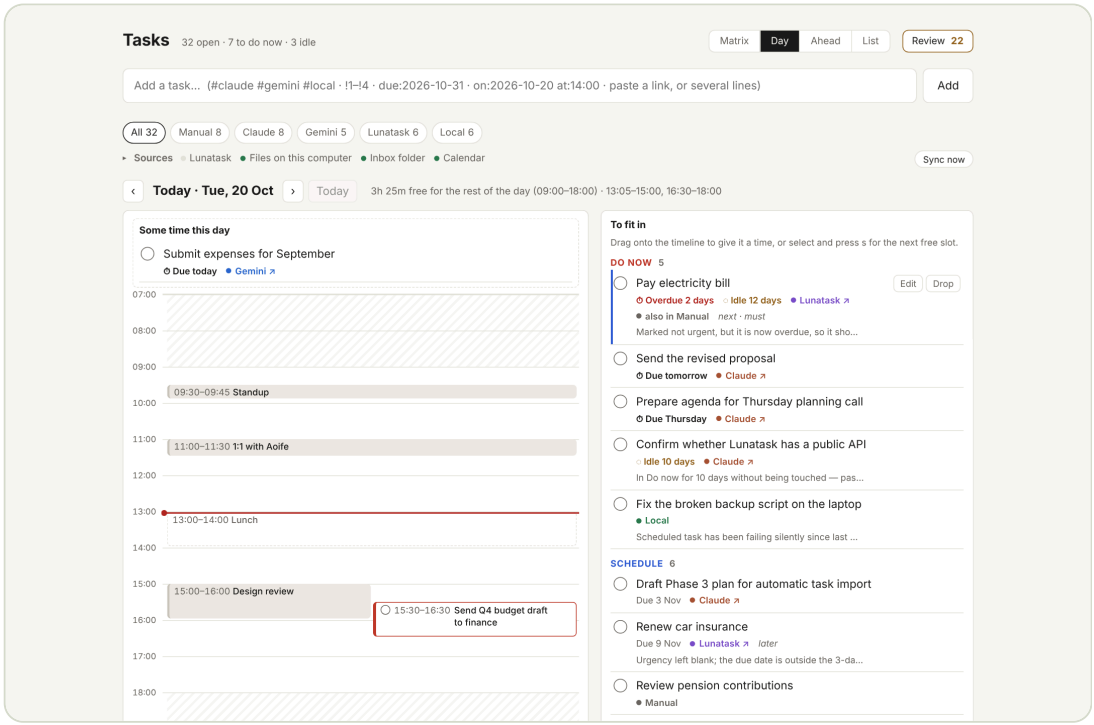
Next: subtasks under a parent task, a settings pane so no text file needs editing, a light and dark switch, a UX review of the whole app as one experience, a phone view, and reporting on what was finished, dropped and left to slide.

06

What it shows

A tool for one person could have been built in an afternoon with none of this. The value of doing it properly is that the method holds at any size: the product owner sets the job, the scope and the never list; decisions are written down where they can be challenged; an AI coding agent does the building, inside those limits, and every change is read; and nothing is trusted until it has been tested by someone trying to break it. That is how I would run an AI-assisted team, and this is a small, complete, public example of it.

The documentation set, kept with the code: context and scope, the job-story backlog, the decision log, a setup and troubleshooting guide, a user guide, architecture and data-model notes, the API, development conventions, and the QA report with its retest.



The Day view, with made-up sample data: the day's calendar on the left, the open tasks still to fit in on the right in matrix order, and the free time left in the header.

Built with: TypeScript on Node.js with no runtime dependencies and no build step; plain browser JavaScript; about 5,700 lines of application code and 1,800 of tests. All rights reserved; the code is public to read at github.com/rorymcclelland/task-manager.

Rory McClelland · product leader, AI products and governance
hello@mcclelland.info · mcclelland.info

Companion case studies:
How I build with AI · **Job Scout**